

AI PDF Copilot - Complete Implementation Guide (5-Day Sprint)

Objective

Build a production-style AI PDF Copilot by Friday and demonstrate it on Saturday. The guide is intentionally written so that another LLM can use it as a complete project specification and generate the required code module-by-module.

Project Specification

- Frontend: Streamlit
- Language: Python
- LLM: Groq (Llama 3 preferred)
- Framework: LangChain
- Vector Database: Chroma
- PDF Loader: PyPDFLoader
- Embeddings: HuggingFace Embeddings
- Project Type: Retrieval-Augmented Generation (RAG) AI Copilot

Final Features

- Upload one or multiple PDFs.
- Automatically extract text, split into chunks and index the content.
- Chat with uploaded PDFs using RAG.
- Generate short, detailed and bullet summaries.
- Semantic search over the document.
- Display page/source citations.
- Explain concepts in beginner-friendly language.
- Generate key takeaways.
- Generate quizzes (MCQ, True/False, Short Answer).
- Generate flashcards.
- Suggest useful questions.
- Display document statistics.
- One-click Study Mode that generates Summary + Key Takeaways + Quiz + Flashcards + Suggested Questions.

Monday - Foundation

Tasks

- Create project folder structure (app.py, loaders/, chains/, prompts/, utils/, vector_store/, assets/).
- Create Python virtual environment.
- Install dependencies.
- Create Streamlit home page and sidebar.
- Connect Groq API.
- Verify a simple LLM prompt-response works.
- Implement PDF upload component.
- Extract text using PyPDFLoader.
- Split text using RecursiveCharacterTextSplitter.
- Deliverable: User can upload PDF and extracted text is available.

End of Day Checklist

- Code committed to GitHub.
- Feature tested with sample PDFs.
- No major runtime errors.
- Ready to continue to next day.

Tuesday - RAG Pipeline

Tasks

- Generate embeddings for chunks.
- Store chunks in Chroma.
- Create retriever.
- Build LangChain Retrieval QA chain.
- Create chat interface.
- Display retrieved page numbers with answers.
- Implement semantic search.
- Deliverable: User chats with uploaded PDF.

End of Day Checklist

- Code committed to GitHub.
- Feature tested with sample PDFs.
- No major runtime errors.
- Ready to continue to next day.

Wednesday - Learning Features

Tasks

- Implement summary prompts (short, detailed, bullet).
- Implement Explain in Simple Words feature.
- Generate Key Takeaways.
- Improve prompts for better formatting.
- Deliverable: Complete learning assistant.

End of Day Checklist

- Code committed to GitHub.
- Feature tested with sample PDFs.
- No major runtime errors.
- Ready to continue to next day.

Thursday - Study Mode

Tasks

- Generate MCQs.
- Generate True/False questions.
- Generate Short Answer questions.
- Generate Flashcards.
- Generate Suggested Questions.
- Display document statistics (pages, words, chunks, reading time).
- Implement Study Mode button that automatically generates Summary, Key Takeaways, Quiz, Flashcards and Suggested Questions.
- Deliverable: Complete AI PDF Copilot functionality.

End of Day Checklist

- Code committed to GitHub.
- Feature tested with sample PDFs.
- No major runtime errors.
- Ready to continue to next day.

Friday - Polish

Tasks

- Improve Streamlit UI.
- Add loading indicators and error handling.
- Test with multiple PDFs.
- Write README.md explaining architecture and setup.
- Create requirements.txt.
- Push code to GitHub.
- Prepare a 5-7 minute demo.
- Deliverable: Production-ready demo.

End of Day Checklist

- Code committed to GitHub.
- Feature tested with sample PDFs.
- No major runtime errors.
- Ready to continue to next day.

Expected Project Structure

- app.py - Streamlit UI
- loaders/pdf_loader.py - PDF extraction
- vector_store/chroma_store.py - Embeddings & Chroma
- chains/rag_chain.py - Retrieval pipeline
- utils/summarizer.py - Summary generation
- utils/quiz_generator.py - Quiz generation
- utils/flashcards.py - Flashcards
- utils/study_mode.py - One-click Study Mode
- prompts/prompts.py - Prompt templates
- requirements.txt and README.md

Demo Flow

- Upload PDF.
- Show document statistics.
- Generate summary.
- Ask questions about the document.
- Explain a concept.
- Demonstrate semantic search with citations.
- Click Study Mode to generate summary, key takeaways, quiz, flashcards and suggested questions.
- Conclude by explaining the RAG pipeline and technologies used.